

动态规划入门 树形动态规划

主讲：黄凤林



➔ 什么是树型动态规划

顾名思义，树型动态规划就是在“树”的数据结构上的动态规划，平时作的动态规划都是线性的或者是建立在图上的，线性的动态规划有二种方向既向前和向后，相应的线性的动态规划有二种方法既顺推与逆推，而树型动态规划是建立在树上的，所以也相应的有二个方向：

1.根—>叶：不过这种动态规划在实际的问题中运用的不多，也没有比较明显的例题，所以不在今天讨论的范围之内。

2.叶 - >根：既根的子节点传递有用的信息给根，完后根得出最优解的过程。这类的习题比较多，下面就介绍一些这类题目和它们的一般解法。



→ 树形DP规律总结

- ✧ 一般来说树形dp在设状态转移方程时都可以用:
 - $dp[i][][]$ 表示以i这个节点为根的子树怎么怎么样的最优解,
 - 实现时一般都是用子树更新父亲（即从下向上更新）。
- ✧ 那么首先应该考虑的是一个一个子树的更新父亲还是把所有子树都算完了在更新父亲？这就要因题而异了，一般来说有两种情况：
 1. 需要把所有子树的信息都掌握之后再更新子树的就需要把所有子树都算完了在更新父亲。
 2. 而像树上背包这样的问题就需要一个一个的更新，每次都用一个子树更新已经更新完的子树+父亲，最后就可以将这一部分的子树更新完了，再继续往上更新，最后根节点就是答案。



→ 树形动规实现方式

✧ 实现方式：记忆划搜索

✧ 主要过程：

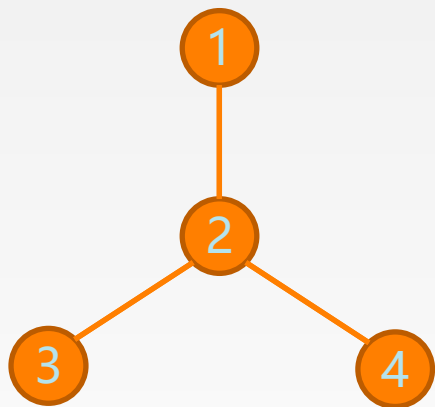
- 1、判断结果（时候已经求出）
- 2、DFS（递归子树）
- 3、回溯更新答案



树形动态规划---树的重心

树的重心

- 对于一颗n个结点的无根树，找到一个点，使得把树变成以该点为根的有根树时，最大子树的结点数最小。换句话说，**删除这个点后最大联通块的结点数最小，那么这个点就是树的重心。如下图，该树的重心即为2号结点**



如何求解树的重心呢？

朴素方法：枚举每一个点，求得以这个点为根的最大子树大小，然后取其最小值即可，时间复杂度($n*n$)

优化方法：设 $siz[i]$ 表示以 i 为根的子树的结点数,任选一个结点为根,把无根树转换成有根树,可知 $siz[i] = \sum_{j \in s[i]} (f[j] + 1)$ 。我们只需要一遍dfs，即可求出 $siz[i]$ ；再分析，在删除结点 i 后最大的联通块有多少个结点呢？这里最大的联通块只能是两个部分，一个是由 i 点下方的子树和 i 点上方的联通块组成， i 点下方子树中最大的有 $\max(siz[j])$ 个结点， i 的“上方子树”中有 $n - siz[i]$ 个结点。在动态规划的过程中就可以根据定义顺便找出树的重心了。



树形动态规划---树的中心

✧ 树的中心:

✧ 给出一棵边带权的树，求树中的点，使得此点到树中的其他节点的最远距离最近

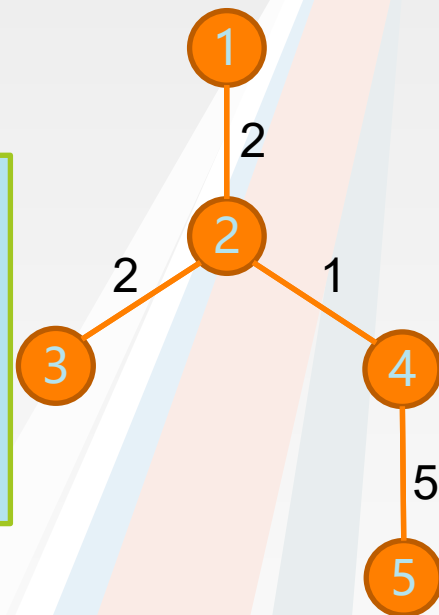
✧ 如右图，4号结点即为树的中心

问题分析：n*n的算法略，

我们分析：从任意一点i出发的最长路径的可能形态有两种。

1从i点出发向上，即终点不在以i为根的子树中的最长路径为u[i]

2从i点出发向下，即终点在以i为根的子树中的最长路径长度为d1[i]



我们分析：从任意一点*i*出发的最长路径的可能形态有两种。

(2)从i点出发向下，即终点在以i为根的子树中的最长路径长度为d[i]

这里关键如何求出 $u[i]$ 。

i向上的路径必经过(i,prt[i]), 而i点的父结点prt[i]又引出两条路径, 一条是prt[i]向上的最长路径, 其长度为u[prt[i]];而另一条是prt[i]向下的路径,该路径不能途径i点, 否则会产生重复计算, 如下图:

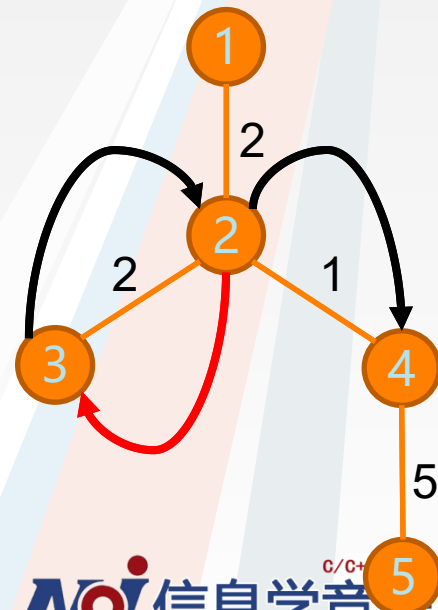
因此设:

$d1[i]$ 表示以 i 为根的子树中， i 到叶子结点的距离最大值；

d2[i]表示以i为根的子树中，i到叶子结点的距离次大值；

分别用 $c1[i]$ 和 $c2[i]$ 记录 $d1[i]$ 和 $d2[i]$ 是从哪个子树更新而来的。

$u[i]$ 表示除了以 i 为根的子树中的叶子结点外，其他的叶子结点到 i 的最大值。





→ 树形动态规划---树的中心

算法实现步骤

1. 首先，一遍树形dp算出 $d1, d2, c1, c2$ ，令 j 是 i 的儿子，则：
 - (1) 若 $d1[j] + dis[i][j] > d1[i]$ ，则 $d2[i] = d1[i]$; $d1[i] = d1[j] + dis[i][j]$;
 - (2) 否则，若 $d1[j] + dis[i][j] > d2[i]$ ，则 $d2[i] = d1[j] + dis[i][j]$;
2. 然后，设 $p1[i] = x$ ，
 - 若 $c1[x] \neq i$ 即 $d1[x]$ 不从 i 更新而来，那么 $u[i] = \max(d1[x], u[x]) + dist[x][i]$;
 - 若 $c1[x] = i$ 即 $d1[x]$ 从 i 更新而来，那么 $u[i] = \max(d2[x], u[x]) + dist[x][i]$ 。
3. 最后在 n 个结点中找出最大值，即
 - $t[i] = \max\{u[i], d1[i]\} \quad (1 \leq i \leq n)$
4. 树的中心： $ans = \min\{t[i]\} \quad (1 \leq i \leq n)$



分题型说树形dp的规律

✧ 1.子树和计数。

这类问题主要是统计子树和，通过加减一些子树满足题目中要求的某些性质。

● 1.codeforces 767C Garland

这道题是让你把树分成3部分，使每部分点权和相等，这就是通过算子树的size[]实现的。

● 2.洛谷1122最大子树和

这道题让你剪去一些子树，让剩下的子树点权和最大。这题就要维护子树的点权和， $dp[i]$ 表示i这颗子树的点权和最大值。



➔ P1122 最大子树和

✧ 问题描述

一株奇怪的花卉，上面共连有 N 朵花，共有 $N-1$ 条枝干将花儿连在一起，并且未修剪时每朵花都不是孤立的。每朵花都有一个“美丽指数”，该数越大说明这朵花越漂亮，也有“美丽指数”为负数的，说明这朵花看着都让人恶心。

所谓“修剪”，意为去掉其中的一条枝条，这样一株花就成了两株，扔掉其中一株。经过一系列“修剪”之后，还剩下最后一株花（也可能是一朵）。

任务就是：通过一系列“修剪”（也可以什么“修剪”都不进行），使剩下的那株（那朵）花卉上所有花朵的“美丽指数”之和最大。

✧ 数据范围

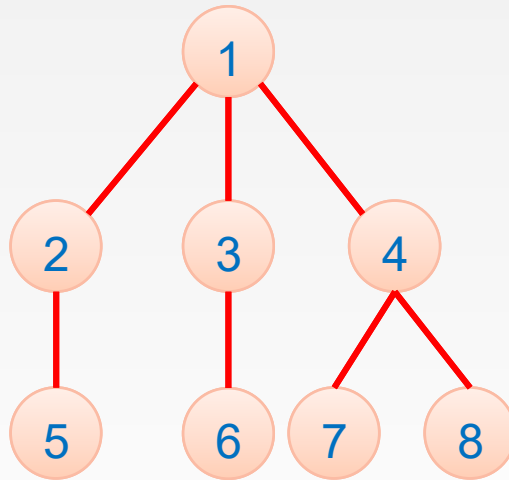
对于 60% 的数据，有 $N \leq 1000$ ；

对于 100% 的数据，有 $N \leq 16000$ 。



→ 问题分析

- ✧ 本题比较简单：
- ✧ 根据题目分析，减去不同的枝条会导致留下的子树值不同。因此用 $dp[i]$ 表示以 i 为根节点的子树的最大美丽值，则：
$$dp[i] = \sum dp[child] + a[i], dp[child] > 0$$
- ✧ 最后，枚举根节点 i ， $ans = \max\{dp[i]\}$ ，即为答案。
- ✧ 时间复杂度 $O(n)$





→ 参考代码

```
int n,a[maxn],dp[maxn];//dp[i]表示以i为根的子树的权值最大值
int dfs(int x,int fa){//计算以x为根的子树的最大值
    if(dp[x]!=inf) return dp[x];//表示已经算出
    dp[x]=a[x];//初始化dp[x]=结点x的值
    for(int i=head[x];i;i=edge[i].next){
        int v=edge[i].to; if(v==fa) continue;
        dfs(v,x);//递归求解x的孩子结点v
        if(dp[v]>0) dp[x]+=dp[v];//回溯修改以x为根子树的dp值
    } return dp[x];
}
int main(){
    int u,v;
    scanf("%d",&n);
    for(int i=1;i<=n;i++) scanf("%d",&a[i]);
    for(int i=1;i<n;i++){
        scanf("%d%d",&u,&v);
        add(u,v);add(v,u);
    }
    for(int i=1;i<=n;i++) dp[i]=inf;
    dfs(1,1);
    int maxx=inf;
    for(int i=1;i<=n;i++) {
        //printf("%d %d\n",i,dp[i]);
        maxx=max(maxx,dp[i]);
    }
    printf("%d\n",maxx);
}
```



分题型说树形dp的规律

✧ 2. 树上背包问题

这类问题就是让你求在树上选一些点满足条件价值最大的问题，一般都可以设 $f[i][j]$ 表示 i 这颗子树选 j 个点的最优解。

● 1. 洛谷1272重建道路

这道题是让你剪去最少的边实现最后剩 P 个点。所以 P 个点就是背包，剪去的边数就是价值。这题需要转化一下把剪边变成加边。

● 2. 洛谷1273有线电视网

这道题是让你在保证不亏损的情况下满足更多的客户看到转播。此时用户的个数就是容量， $f[i][j]$ 表示 i 这颗子树选 j 个用户能赚的最多的钱。



➔ 愚蠢的矿工

✧ 问题描述

这个宝藏的制造者为了掩盖世人耳目,他做的宝藏是没有出口,只有入口,不少建造宝藏的人都死在里面.现在知道宝藏总共有 N 个分岔口,在分岔口处是有财宝的,每个宝藏点都有一个财富值.狗狗只带了 M 个人来,而且为了安全起见,在每个分岔口,必须至少留一个人下来,这个留下来的人可以挖宝藏(每个人只能挖一个地方的宝藏),这样才能保证不会迷路,而且这个迷宫有个特点,任意两点间有且只有一条路可通。

狗狗为了让他的00开心,决定要尽可能地多挖些宝藏回去.现在狗狗的圈叉电脑不在身旁,只能求救于你了,你要知道,狗狗的终身幸福就在你手上了。



➔ 愚蠢的矿工

✧ 输入格式：

第1行：两个正整数 N, M 。 N 表示宝藏点的个数, M 表示狗狗带去的人数(那是一条懒狗,他自己可不做事)。($n \leq 1000, m \leq 100$)

第2行： N 个整数,第 i 个整数表示第 i 个宝藏的财富值($|w_i| < \text{maxint}$)。

第 $N+2$ 行：两个非负整数 A 和 B ，表示 A 通向 B ,当 $A=0$,表示 A 是入口。(保证 $A, B \leq n$)

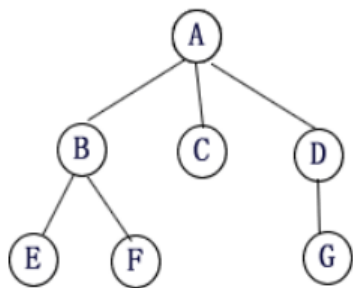
✧ 输出格式：输出狗狗能带回去的宝藏的价值。

样例输入：	样例输出：
4 3	13
5 6 2 4	
1 2	
0 1	
2 3	
3 4	



→ 动态规划

- ✧ 通过“任意两点间有且只有一条路可通”这句话，我们可以知道这个迷宫是一个树形结构。
- ✧ **分析：**DP思路，假设给定如下地图，安排进3人，如何才能使得挖到的宝藏最多，如下图：



分析发现：可选的方案有：
ABE、ABF、ABC、ABD、ACD、ADG

用 $dp[i][j]$ 表示：以节点 i 为根的子树派 j 个人得到的最大宝藏
则有： $dp[i][j] = \max(dp[c_1][k_1], dp[c_2][k_2], \dots, dp[c_i][k_i])$

答案为： $dp[\text{入口}][m]$

实现方式：记忆化搜索



➔ 问题分析

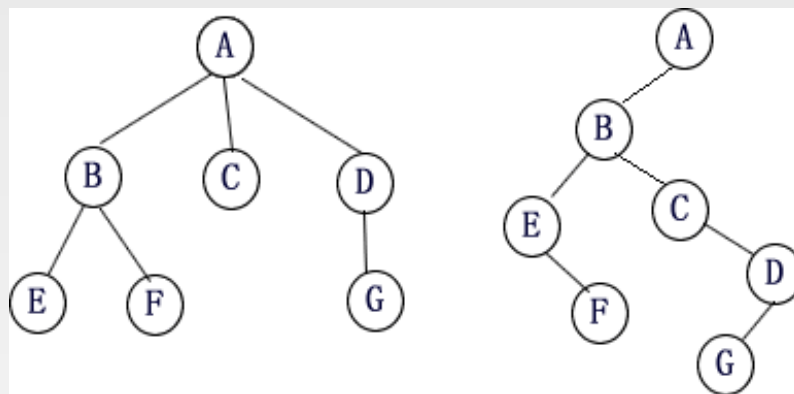
通过“任意两点间有且只有一条路可通”这句话，我们可以知道这个迷宫是一个树形结构，但该树形结构是一个多叉树，且分配人数也是一个枚举变量，因此如果用**一个多叉树 + 枚举变量递归枚举每一种情况**，是一个十分难做的问题，如我们的dp方程，那么怎么办呢？

**多叉树转二叉树
二叉树如何处理？**



→ 多叉树转二叉树

左孩子右兄弟

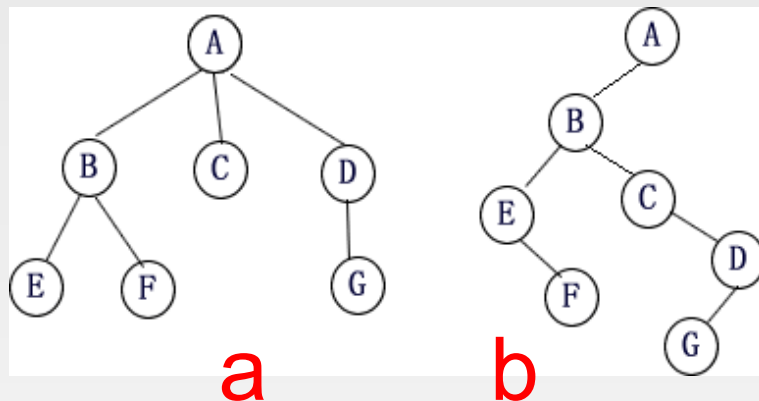


```
int vis[1002]; // vis[i] 表示是否有孩子相连
for(int i=1; i<=n; i++){
    scanf("%d%d", &u, &v);
    if(!vis[u]) tree[u].left=v; // 没有孩子
    else tree[vis[u]].right=v;
    vis[u]=v;
}
```

*上面通过读入将一个多叉树转换成二叉树，请认真体会



→ 多叉转成二叉的差异



- ✧ 如果给定的树是a图结构，转换成二叉树为b图结构；**观察会发现二叉树的左孩子为原树形结构的孩子节点；转化后二叉树的右孩子为原树形结构的兄弟节点。**
- ✧ 加入D点有宝藏，B,C都没有。而安排进去的人只有2个。那么我们按照题目意思，这里可以选择的方案可以是A—B，A—C,A—D,当然我们要选择的是A—D是最优的。
- ✧ 那么b图如何做才能实现这种效果呢？按照题目的意思，如果根节点往子树节点，每次减少一个人（按题意占位置），则始终只能到达B点（因为向下已经没有人可用了，则方案只能是A--B），而C点、D点永远达到不了，将导致结果错误。
- ✧ 那怎么办呢？是否有办法可以解决这个问题？



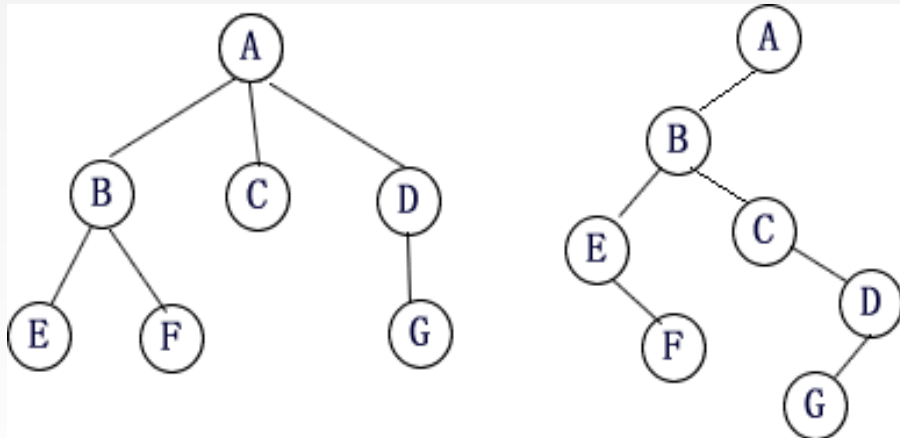
→ 多叉转成二叉的差异

✧ 答案当然是有的，关键是如何做？

通过分析发现，如果把转化好的二叉树从根往子树递归，每次均减少一个人的做法是不能达到和多叉树一样效果。

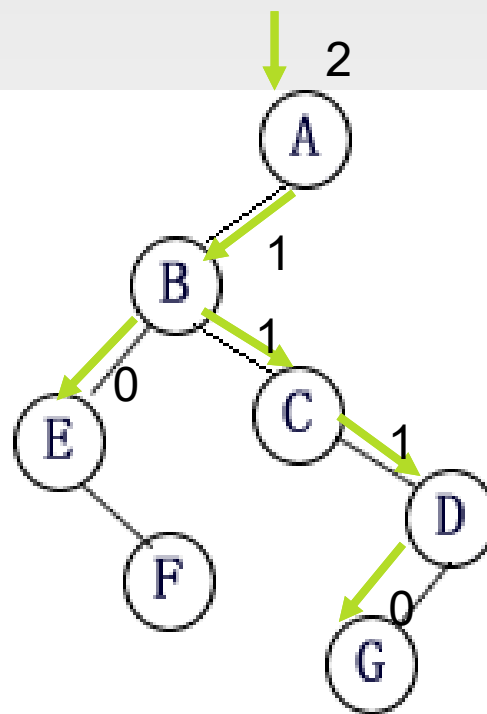
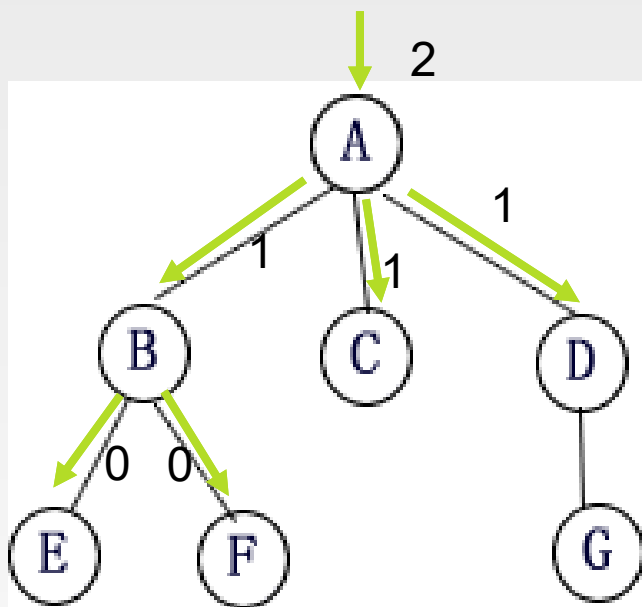
为了达到转换后的二叉树与多叉树效果一致，我们需要**先往右孩子递归，且人数不能减少；再往左孩子递归，人数减少一（必须占用）**。

这是为什么呢？因为**左孩子是孩子，而右孩子是兄弟**





➔ 图示分析





→ 参考代码

```
int dfs(int u,int x){//表示给u这个节点分配x个人取得的最优解
    if(x==0 || u==0) return 0;
    if(dp[u][x]!=0) return dp[u][x];
    dp[u][x]=dfs(tree[u].right,x);//递归右孩子, 且人数不减少,
    for(int i=1;i<=x;i++){//枚举人数分配
        int t1=dfs(tree[u].left,i-1);
        int t2=dfs(tree[u].right,x-i);
        if(dp[u][x]<t1+t2+a[u]) dp[u][x]=t1+t2+a[u];
    }
    return dp[u][x];
}
```

```
for(int i=1;i<=n;i++){
    scanf("%d%d",&u,&v);
    if(!vis[u]) tree[u].left=v;//没孩子
    else tree[vis[u]].right=v;//已有孩子, 则新的节
    vis[u]=v;
    if(u==0) root=u;
}
int ans=dfs(tree[root].left,m);
printf("%d\n",ans);
```



分题型说树形dp的规律

✧ 3. 花费最少的费用覆盖所有点

这类问题是父亲与孩子有联系的题。基本有两种出法：

1. 选父亲必须不能选孩子（强制）
2. 选父亲可以不用选孩子（不强制）。

● 1. 洛谷2458 [SDOI2006]保安站岗

这题属于类型2，需要预估，要不要选这个节点的父亲。

$f[i][0]$ 表示选自己， $f[i][1]$ 表示选孩子， $f[i][2]$ 表示选父亲

● 2. UVA 1220 Party at Hali-Bula

（这是最大独立集问题，用做和上面题区分）

这题就是强制要求父亲和孩子不能同时选，但这题没有要求必须把所有点完全覆盖，只是让选尽可能多的点，所以这样就可以用 $f[i][0]$ 表示选自己， $f[i][1]$ 表示选孩子，省去 $f[i][2]$ 表示选父亲了，因为没有都覆盖的强制要求，他的父亲选不选可以到他父亲再判断。



→ P1352 没有上司的舞会

✧ 问题描述

某大学有 N 个职员，编号为 $1 \sim N$ 。他们之间有从属关系，也就是说他们的关系就像一棵以校长为根的树，父结点就是子结点的直接上司。现在有个周年庆宴会，宴会每邀请来一个职员都会增加一定的快乐指数 R_i ，但是呢，如果某个职员的上司来参加舞会了，那么这个职员就无论如何也不肯来参加舞会了。所以，请你编程计算，邀请哪些职员可以使快乐指数最大，求最大的快乐指数。

✧ 数据范围： $(1 \leq N \leq 6000)$



→ 动态规划分析

- ✧ 用 $dp[i][1]$ 表示以 i 为根的子树且选择 i 的最大快乐值
- ✧ 用 $dp[i][0]$ 表示以 i 为根的子树不选择 i 的最大快乐值
- ✧ 则有： k 属于 i 的孩子
- ✧ $dp[i][1] = R[i] + \sum_{k=1}^n dp[k][0]; dp[k][0] > 0;$
- ✧ $dp[i][0] = \sum_{k=1}^n \max(dp[k][0], dp[k][1]);$



→ 参考代码

```
int dp[maxn][2], a[maxn];  
//dp[i][1]表示以i为根的子树且选择i的最大快乐值  
//dp[i][0]表示以i为根的子树不选择i的最大快乐值  
int n, root, f[maxn];  
int cnt=0;  
void dfs(int x){  
    if(dp[x][1]!=inf) return;  
    dp[x][1]=a[x], dp[x][0]=0;  
    for(int i=head[x]; i>0; i=edge[i].next){  
        int v=edge[i].to; dfs(v);  
        if(dp[v][0]>0) dp[x][1]+=dp[v][0];  
        dp[x][0]+=max(dp[v][0], dp[v][1]);  
    }  
}
```



→ 警卫安排

- ✧ 一个有N个区域的树结构上需要安排若干个警卫;
- ✧ 每个区域安排警卫所需要的费用是不同的;
- ✧ 每个区域的警卫都可以望见其相邻的区域, 只要一个区域被一个警卫望见或者是安排有警卫, 这个区域就是安全的;
- ✧ 任务: 在确保所有区域都是安全的情况下, 找到安排警卫的最小费用;
- ✧ $0 < n \leq 720$;



→ 问题分析

✧ 分析样例

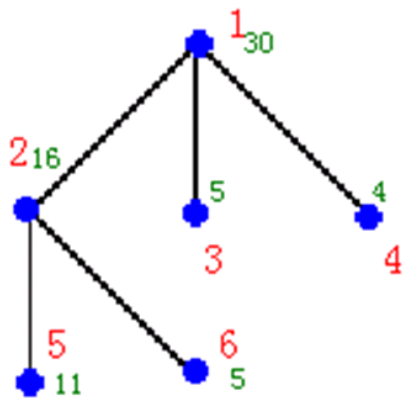


图1

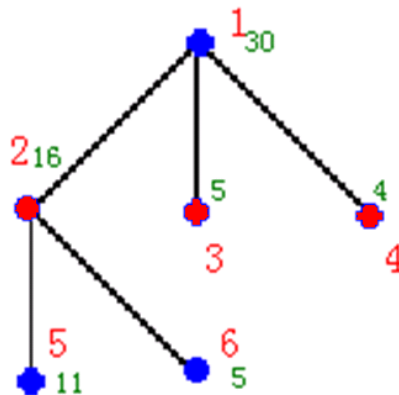


图2

✧ 该图有6个区域如图1，安排情况如图2，红色点为安排了警卫。

2号警卫可以观察1，2，5，6；

3号警卫可以观察1，3；

4号警卫可以观察1，4；

✧ 费用： $16+5+4=25$



→ 问题分析

✧ 对于每个点 i ，都有3种状态分别为：

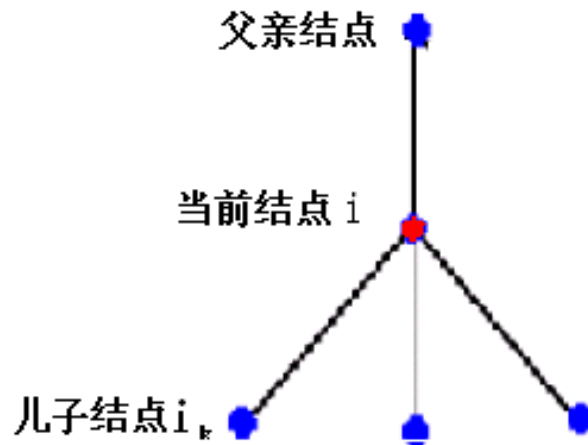
- 要么在父亲结点安排警卫，即被父亲看到
- 要么在儿子结点安排警卫，即被儿子看到
- 要么安排警卫

✧ 对于 i

- i 被父亲看到，这时 i 没有安排警卫， i 的儿子要么安排警卫，要么被它的后代看到。
- i 被儿子看到，即 i 的某个儿子安排了警卫，其他儿子需要安排警卫或者被它的后代看到。
- i 安排了警卫， i 的儿子可能还需要安排警卫，这样可能有更便易的方式照顾到它的后代；所以 i 的儿子结点被 i 看到，可能安排警卫，可能被它的后代看到。



→ 问题分析



(1) 被父亲看到 对各儿子来说： 1. 要么被儿子看到 2. 要么安排警察	(2) 被儿子看到 对各儿子来说： 1. 一定有某个儿子安排了警察 2. 其他儿子要么安排警察 要么被儿子看到	(3) 安排警察 对各儿子来说： 1. 儿子一定被父亲看到 2. 儿子要么安排警察 要么被儿子看到
--	---	---



→ 动态规划

✧ 根据上述三种情况，可以设计出一个二维状态，记录在状态下所需要最少的保安人数，则：

1. $dp[u][0]$: u 节点由子节点照看
2. $dp[u][1]$: u 节点由本身照看
3. $dp[u][2]$: u 节点由父节点照看

✧ 状态转移方程

$$1. dp[u][1] = \sum(\min(dp[v][0], dp[v][1], dp[v][2])) + a[u]$$

$$2. dp[u][2] = \sum(\min(dp[v][0], dp[v][1]))$$

3. $dp[u][0]$ 这个比较复杂，因为必须至少选一个子节点中的 $dp[v][1]$ ，但有可能 $dp[v][1] < dp[v][0]$ ，所以具体的办法就是比较这两个的大小，如果 $dp[v][1] < dp[v][0]$ ，选择 $dp[v][1]$ ，后面就不需要再从原来的里面选一个 $dp[v][1]$ 了。如果一个 $dp[v][1]$ 都没选，就选择一个 $dp[v][1] - dp[v][0]$ 最小的差，加在原来 $dp[u][0]$ 上。



→ 参考代码

```
void dfs(int u,int fa)
{
    dp[u][1]=a[u];
    dp[u][0]=dp[u][2]=0;
    int cnt=0;
    int cha=0x3f3f3f3f;
    for(int i=head[u];i;edge[i].next)
    {
        int v=edge[i].to
        if(v==fa)continue;
        dfs(v,u);
        dp[u][2]+=min(dp[v][1],dp[v][0]);
        dp[u][1]+=min(min(dp[v][1],dp[v][0]),dp[v][2]);
        if(dp[v][1]<dp[v][0]) cnt++;
        else cha=min(cha,dp[v][1]-dp[v][0]);
        dp[u][0]+=min(dp[v][1],dp[v][0]);
    }
    if(cnt==0)
        dp[u][0]+=cha;
}
```




➔ 保安站岗(P2458)

✧ 题目描述

五一来临，某地下超市为了便于疏通和指挥密集的人员和车辆，以免造成超市内的混乱和拥挤，准备临时从外单位调用部分保安来维持交通秩序。

已知整个地下超市的所有通道呈一棵树的形状；某些通道之间可以互相望见。总经理要求所有通道的每个端点（树的顶点）都要有人全天候看守，在不同的通道端点安排保安所需的费用不同。

一个保安一旦站在某个通道的其中一个端点，那么他除了能看守住他所站的那个端点，也能看到这个通道的另一个端点，所以一个保安可能同时能看守住多个端点(树的结点)，因此没有必要在每个通道的端点都安排保安。



分题型说树形dp的规律

✧ 4. 树上统计方案数问题

这类问题就是给你一个条件，问你有多少个点的集合满足这样的条件。这类题主要运用乘法原理，控制一个点不动，看他能做多少贡献。

1. 51nod 1588 幸运树。

问有多少个三元组满足幸运数字，可以控制一个点不动，分成子树内，子树外两个部分，分别相乘就行了。

可以看这个博客。



→ 幸运树

✧ 问题描述

比得喜欢幸运数字。这里所说的幸运数字是由4和7组成的正整数。

比如：数字47，744，4是幸运数字，而5，17，467就不是。

一天，比得遇到一棵由 n 个点组成的树。另外，这棵树是带权的，即每条边有一个权值（由一个正整数表示）。

如果一条边的权值是一个幸运数字，那么我们就说这条边是一条幸运边。说明一下，一棵 n 个结点的树是由 n 个结点和 $n-1$ 条边组的无环的无向图。

比得好奇，在树中有多少个满足以下条件的三元组 $tr(i,j,k)$ (i,j,k 是三个不同的点)。

- 1、 i 到 j 有路径， i 到 k 也有路径。
- 2、每条路径中至少有一条幸运边。

数字的顺序是有意义的，举例说明， $tr(1,2,3), tr(1,3,2), tr(2,1,3)$ 是三个不同的序列。

现在要求计算在树中存在多少个这样的三元组关系。



→ 幸运树

✧ 输入：第一行包含一个整数 n ($1 \leq n \leq 10^5$)。

接下来的 $n-1$ 行中每行有三个整数 $u_i \ v_i \ w_i$ ($1 \leq u_i, v_i \leq n, 1 \leq w_i \leq 10^9$) 分别表示有边相连的两个点和这条边的权值。

✧ 输出：共一行，表示题目中所要计算三元组的个数。

✧ 输入样例

4

1 2 4

3 1 2

1 4 7

✧ 输出样例：16

✧ 样例解释：样例一中的16种情况分别为：

$(1,2,4), (1,4,2), (2,1,3), (2,1,4), (2,3,1), (2,3,4), (2,4,1), (2,4,3), (3,2,4), (3,4,2),$
 $(4,1,2), (4,1,3), (4,2,1), (4,2,3), (4,3,1), (4,3,2)$ 。



→ 问题分析

- ✧ 要想不重不漏地枚举所有组合，我们枚举所有的 i ，对于每一个 i ，找到 i 所对应的 (j, k) 的组合数，加起来就行了。

令 $f[u]$ 为在 u 的子树内，到 u 的路径中有幸运边的点有几个，

令 $g[u]$ 为在 u 的子树外，到 u 的路径中有幸运边的点有几个。

- ✧ 那么 (u, j, k) 的方案数 $= f[u] * (f[u] - 1) + g[u] * (g[u] - 1) + f[u] * g[u] * 2$ 转移：

- ✧ 边 (u, v) 是幸运边： $f[u] += \text{size}[v]$ 边 (u, v) 是幸运边：

$f[u] += \text{size}[v]$ 边 (u, v) 不是幸运边；

$f[u] += f[v]$ 边 (u, v) 不是幸运边；

$f[u] += f[v]$ 边 (fa, u) 是幸运边；

$g[u] = \text{size}[1] - \text{size}[u]$ (1是根)边 (fa, u) 是幸运边；

$g[u] = \text{size}[1] - \text{size}[u]$ (1是根)边 (fa, u) 不是幸运边： $g[u] = g[fa] + f[fa] - f[u]$



➔ 加分二叉树 (NOIP2003提高组第三题)

【问题描述】

设一个 n 个节点的二叉树 $tree$ 的中序遍历为 $(1, 2, 3, \dots, n)$ ，其中数字 $1, 2, 3, \dots, n$ 为节点编号。每个节点都有一个分数（均为正整数），记第 i 个节点的分数为 d_i ， $tree$ 及它的每个子树都有一个加分，任一棵子树 $subtree$ （也包含 $tree$ 本身）的加分计算方法如下：

$subtree$ 的左子树的加分 $\times$ $subtree$ 的右子树的加分 $+$ $subtree$ 的根节点的分数

若某个子树为空，规定其加分为1，叶子的加分就是叶节点本身的分数；不考虑它的空子树

试求一棵符合中序遍历为 $(1, 2, 3, \dots, n)$ 且加分最高的二叉树 $tree$ 。要求输出：

(1) $tree$ 的最高加分 (2) $tree$ 的前序遍历

【输入格式】

第1行：一个整数 n ($n < 30$)，为节点个数。

第2行： n 个用空格隔开的整数，为每个节点的分数（分数 < 100 ）。

【输出格式】

第1行：一个整数，为最高加分（结果不会超过4,000,000,000）。

第2行： n 个用空格隔开的整数，为该树的前序遍历。

【输入样例】

```
5
5 7 1 2 10
```

【输出样例】

```
145
3 1 2 4 5
```



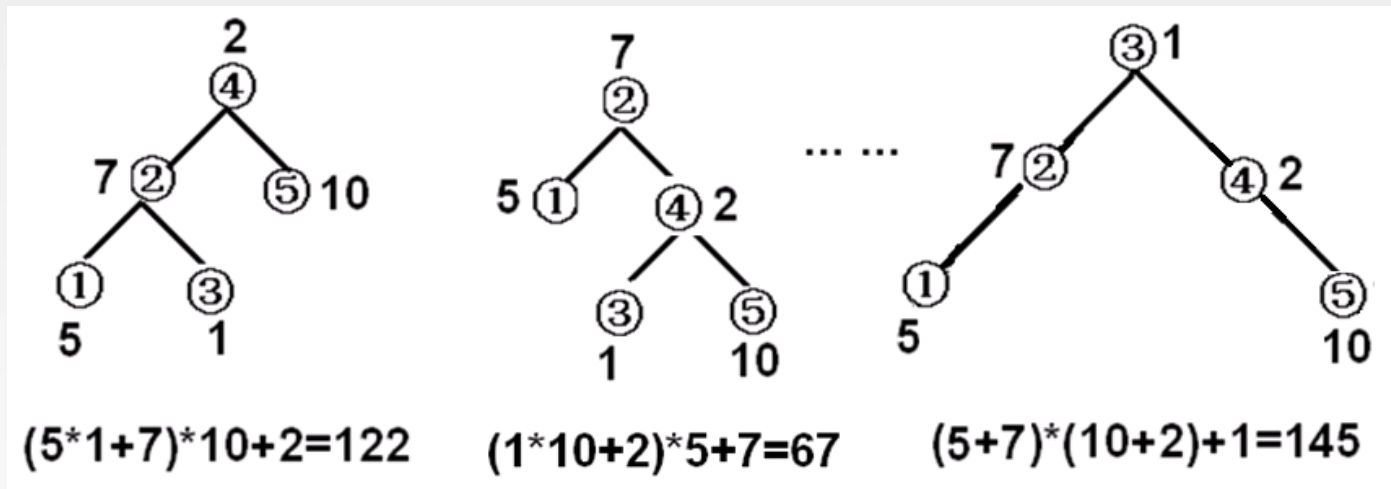
➡ 加分二叉树

- ✧ 给定一个中序遍历为 $1, 2, 3, \dots, n$ 的二叉树
- ✧ 每个结点有一个权值
- ✧ 定义二叉树的加分规则为：
 - 左子树的加分 $\times$ 右子树的加分 $+$ 根的分值；
 - 若某个树缺少左子树或右子树，规定缺少的子树加分为1
- ✧ 构造符合条件的二叉树
 - 该树加分最大
 - 输出其前序遍历序列



➔ 加分二叉树

- ✧ 样例
- ✧ 中序遍历为1,2,3,4,5的二叉树有很多;
- ✧ 下图是其中的三棵, 其中第三棵加分最大, 为145。

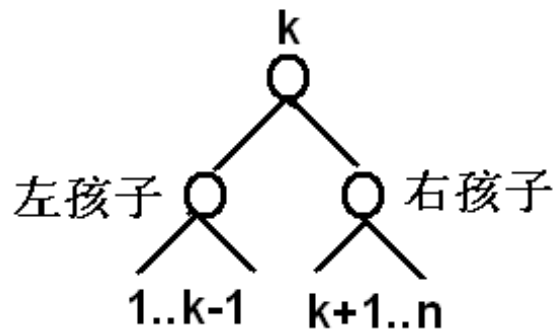




➔ 加分二叉树

- ✧ 性质：中序遍历是按“左-根-右”方式进行遍历二叉树，因此二叉树左孩子遍历序列一定在根结点的左边，右孩子遍历序列一定在根结点的右边！
- ✧ 因此，假设二叉树的根结点为 k ，那么中序遍历为 $1, 2, \dots, n$ 的遍历序列，左孩子序列为 $1, 2, \dots, k-1$ ，右孩子序列为 $k+1, k+2, \dots, n$ ，如下图

$1..k-1$	k	$k+1..n$
左孩子	根	右孩子





→ 动态规划

- ✧ 设 $dp[i][j]$ 表示：中序遍历为 $i, i+1, \dots, j$ 的二叉树的最大加分
 - ◆ 则有： $dp[i][j] = \max\{dp[i][k-1] * f[k+1][j] + d[k]\}$; k 为根
 - ◆ 显然 $dp[i][i] = d[i]$, 答案为 $dp[1][n]$
 - ◆ $1 \leq i \leq k \leq j \leq n$, 时间复杂度 $O(n^3)$
- ✧ 要构造这个树，只需记录每次的决策值，令 $b[i][j] = k$, 表示中序遍历为 $i, i+1, \dots, j$ 的二叉树的取最优决策时的根结点为 k 。
- ✧ 最后前序遍历这个树即可。



→ 参考代码

```
long long dp[maxn][maxn]; // dp[i][j] 表示从节点i到节点j所组成的二叉树的最大加分
int root[maxn][maxn]; // 记录i到j序列中的根节点
int n;
long long dfs(int left, int right){
    if(left > right){ // 空子树的情况
        root[left][right] = 0;
        return dp[left][right] = 1;
    }
    if(dp[left][right] != -1) return dp[left][right]; // 已求出，记忆化搜索
    if(left == right){ // 叶子结点
        root[left][right] = right;
        return dp[left][right] = tree[left];
    }
    dp[left][right] = 0;
    for(int i = left; i <= right; i++) // 用根分出两个部分，去求出left到right中的最大值
    {
        dfs(left, i-1); // i为根，求其左子树的最大加分
        dfs(i+1, right); // i为根，求右子树的最大加分
        if(dp[left][i-1]*dp[i+1][right]+tree[i] > dp[left][right]) // 比较找一个最大加分二叉树
        {
            dp[left][right] = dp[left][i-1]*dp[i+1][right]+tree[i];
            root[left][right] = i;
        }
    }
    return dp[left][right];
}
```



→ 参考代码

```
void print(int left,int right)//打印树的前序。
{
    if(left>right) return;
    printf("%d ",root[left][right]);
    print(left,root[left][right]-1);
    print(root[left][right]+1,right);
}
int main(){
    memset(dp,-1,sizeof(dp));
    scanf("%d",&n);
    for(int i=1;i<=n;i++) scanf("%d",&tree[i]);
    dfs(1,n);
    printf("%d\n",dp[1][n]);
    print(1,n);
    return 0;
}
```



→ P2014: 选课

✧ 问题描述

在大学里每个学生，为了达到一定的学分，必须从很多课程里选择一些课程来学习，在课程里有些课程必须在某些课程之前学习，如高等数学总是在其它课程之前学习。现在有 N 门功课，每门课有个学分，每门课有一门或没有直接先修课（若课程 a 是课程 b 的先修课即只有学完了课程 a ，才能学习课程 b ）。一个学生要从这些课程里选择 M 门课程学习，问他能获得的最大学分是多少？

✧ 输入格式

第一行有两个整数 N, M 用空格隔开。（ $1 \leq N \leq 200, 1 \leq M \leq 150$ ）
接下来的 N 行，第 $i+1$ 行包含两个整数 k_i 和 s_i ， k_i 表示第 i 门课的直接先修课， s_i 表示第 i 门课的学分。若 $k_i=0$ 表示没有直接先修课（ $1 \leq k_i \leq N, 1 \leq s_i \leq 20$ ）。

✧ 输出格式

只有一行，选 M 门课程的最大得分。



→ P2014: 选课

✧ 题目大意

- ✧ 给定M门课程，每门课程有一个学分
- ✧ 要从M门课程中选择N门课程，使得学分总和最大
- ✧ 其中选择课程必须满足以下条件：
 - 每门课程最多只有一门直接先修课
 - 要选择某门课程，必须先选修它的先修课
- ✧ $M, N \leq 500$

✧ 样例输入

```
7 4
2 2
0 1
0 4
2 1
7 1
7 6
2 2
```

✧ 样例输出

```
13
```



→ 分析

- ✧ 每门课程最多只有1门直接先修课，如果我们把课程看成结点，也就是说每个结点最多只一个前驱结点。
- ✧ 如果把前驱结点看成父结点，换句话说，每个结点只有一个父结点。显然具有这种结构的模型是树结构，要么是一棵树，要么是一个森林。
- ✧ 这样，问题就转化为在一个M个结点的森林中选取N个结点，使得所选结点的权值之和最大。同时满足每次选取时，若选儿子结点，必选根结点的条件。



→ 样例分析

- ✧ 如图1：为两棵树，我们可以虚拟一个结点，将这些树连接起来，那么森林就转会为了1棵树，选取结点时，从每个儿子出发进行选取。
- ✧ 显然选 $M=4$ 时，选3, 2, 7, 6四门课程最优。

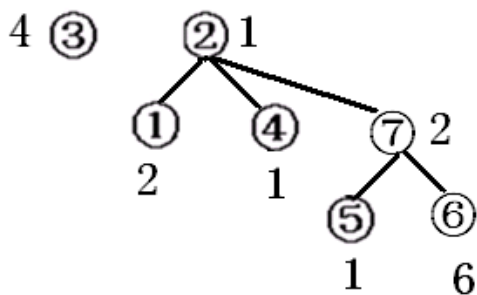


图1

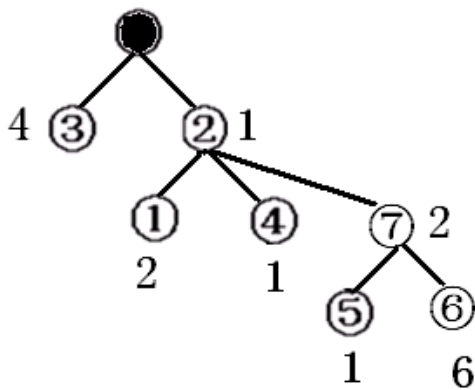
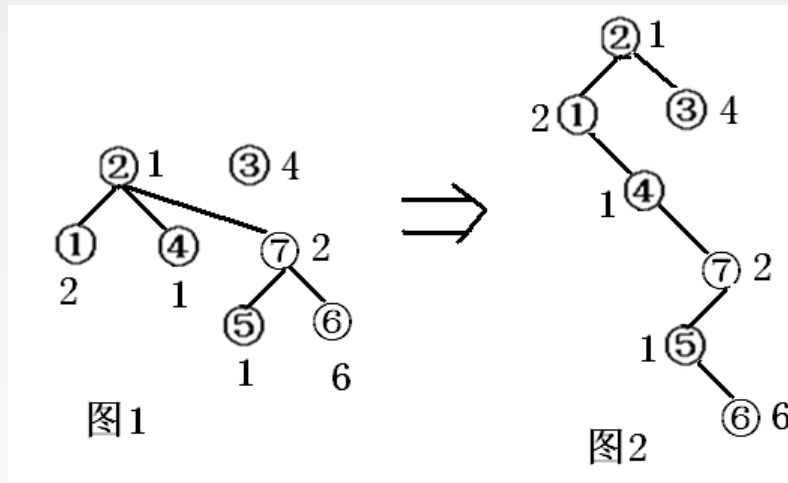


图2



➔ 转化为二叉树

✧ 如果该问题仅仅只是一棵二叉树，我们对儿子的分配就仅仅只需考虑左右孩子即可，问题就变得很简单了。因此我们试着将该问题转化为二叉树求解。



✧ 图2就是对图1采用孩子兄弟表示法所得到的二叉树



→ 动态规划

✧ 仔细理解左右孩子的意义（如右图）：

左孩子：原根结点的孩子

右孩子：原根结点的兄弟

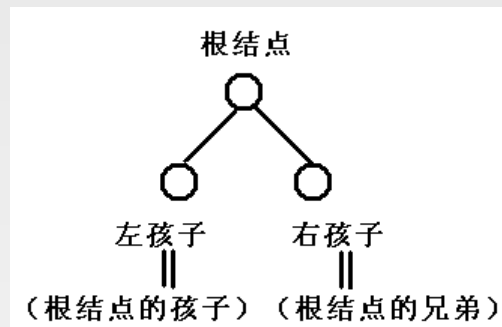
✧ 也就是说，左孩子分配选课资源时，
根结点必须要选修，而与右孩子无关。

✧ 因此，我们设 $f(i, j)$ 表示以 i 为根结点的二叉树分配 j 门课程的所获得的最大学分，则有：

$$f(i, j) = \max \begin{cases} f(i_l, k) + f(i_r, j - k - 1) + a[i], & \text{根结点选修} \\ f(i_r, j), & \text{根结点不选修} \end{cases}$$

✧ $0 \leq k < j < n, i \in (1..m)$

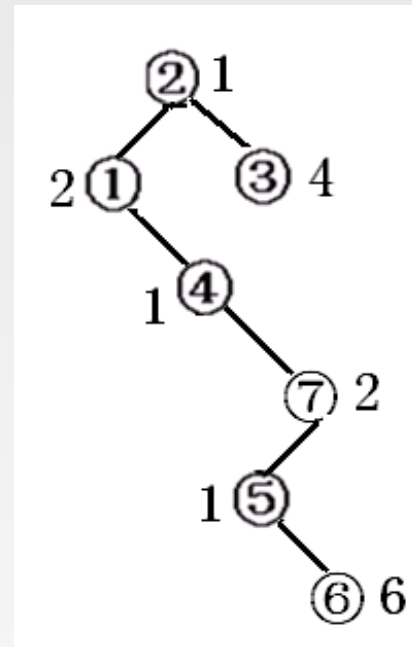
✧ 时间复杂度 $O(mn^2)$





→ 动态规划

- ✧ 样例求解过程：初始 $f(i,0)=0$
- ✧ $f(6,1)=6, f(5,1)=\max\{1,6\}=6, f(7,1)=2$
 $f(4,1)=\max\{1,2\}=2, f(1,1)=\max\{1,f(4,1)\}=2$
 $f(3,1)=4, f(2,1)=\max\{1,4\}=4$
- ✧ $f(5,2)=7$
 $f(7,2)=\max\{f(5,1)+2\}=8$
 $f(4,2)=\max\{f(7,2), f(7,1)+1\}=8$
 $f(1,2)=\max\{f(4,2), f(4,1)+2\}=8$
 $f(2,2)=\max\{f(1,1)+1, f(3,1)+1\}=5$
- ✧ $f(7,3)=9$
 $f(4,3)=\max\{f(7,2)+1, f(7,3)\}=9$
 $f(1,3)=\max\{f(4,2)+1, f(4,3)\}=9$
 $f(2,3)=\max\{f(1,1)+f(3,1)+1, f(1,2)+1\}=9$
- ✧ $f(2,4)=\max\{f(1,3)+1,$
 $f(1,2)+f(3,1)+1\}=\max\{9+1, 8+4+1\}=13$





→ 参考代码

//读入数据,转化为孩子兄弟表示

```
fin >> n >> m;
```

```
score[n+1] = 0;
```

```
brother[n+1] = 0;
```

// 输入数据并转化为左儿子右兄弟表示法

```
for (int i=1; i<=n; ++i) {
```

```
    int a, b;
```

```
    fin >> a >> b;
```

```
    if (a == 0)    a = n + 1;
```

```
    score[i] = b;
```

```
    brother[i] = child[a];
```

```
    child[a] = i;
```

```
}
```



→ 参考代码

```
void dfs( int i, int j){  
    if (visited[i][j]) return;  
    visited[i][j] = 1;  
    if (i==0 || j==0) return;  
    dfs(brother[i], j); // 如果不选i, 则转移到状态(brother[i], j)  
    f[i][j] = f[brother[i]][j];  
    for (int k=0; k<j; ++k) {  
        // 选择i, 枚举学习多少以i为根的 (j-k-1), 并转移到相应状态  
        dfs(brother[i], k); dfs(child[i], j-k-1);  
        // 更新答案  
        if (f[brother[i]][k] + f[child[i]][j-k-1] + score[i] > f[i][j])  
            f[i][j] = f[brother[i]][k] + f[child[i]][j-k-1] + score[i];  
    }  
}
```



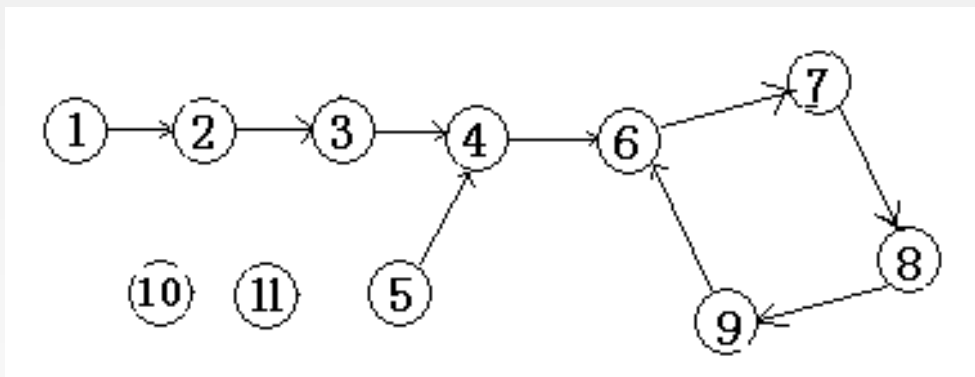
→ 软件安装(2010河南省选)

- ✧ 有 N 个软件，对于一个软件 i ，它要占用 W_i 的磁盘空间，它的价值为 V_i 。我们希望能从中选择一些软件安装到一台磁盘容量为 M 的计算机上，使得这些软件的价值尽可能大（即 V_i 的和最大）。
- ✧ 软件之间存在依赖关系，即软件 i 只有在安装了软件 j （包括软件 j 的直接或间接依赖）的情况下才能正确工作（软件 i 依赖软件 j ）。幸运的是，一个软件最多依赖另外一个软件。如果一个软件不能正常工作，那么它能够发挥的作用为0。
- ✧ 我们现在知道了软件之间的依赖关系：软件 i 依赖软件 D_i 。一个软件只能被安装一次，如果一个软件没有依赖则 $D_i=0$ ，这时只要这个软件安装了，它就能正常工作。
- ✧ 现在请你设计出一种方案，安装价值尽量大的软件。



→ 问题分析

- ✧ 由于软件存在先后约束关系，因此简单按软件先后顺序进行动态规划，会不符合无后效应原理，因此我们需要在进行动态规划前进行预处理。
- ✧ 若安装软件*i*必须先安装*j*，则从*i*向*j*连一条有向弧，则软件的约束关系就构成了一个有向图。如下图：



- ✧ 可以看出如果有*k*个制约关系，则有*k*条边，中间会存在环



➔ 问题分析

➤ 处理环：

- ✧ 由于环为互为前提，要选择环中的一个必须都进行选择，因此可以将环缩成一个点，这个点为权值和价值为其他点的和。
- ✧ 孤立点没有与其他点也没有任何关系，可以不管。
- ✧ 如果把每个连通分量看成一棵树，则图变成了为一个森林，图2：

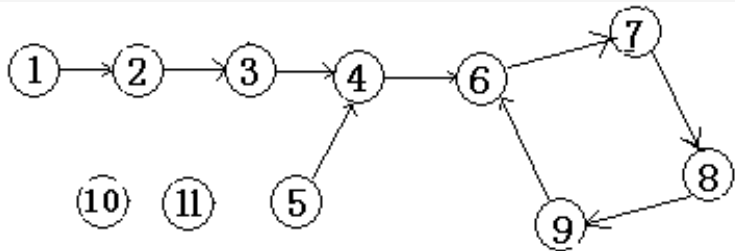


图1

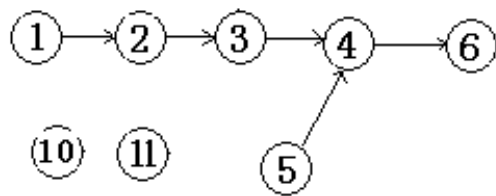
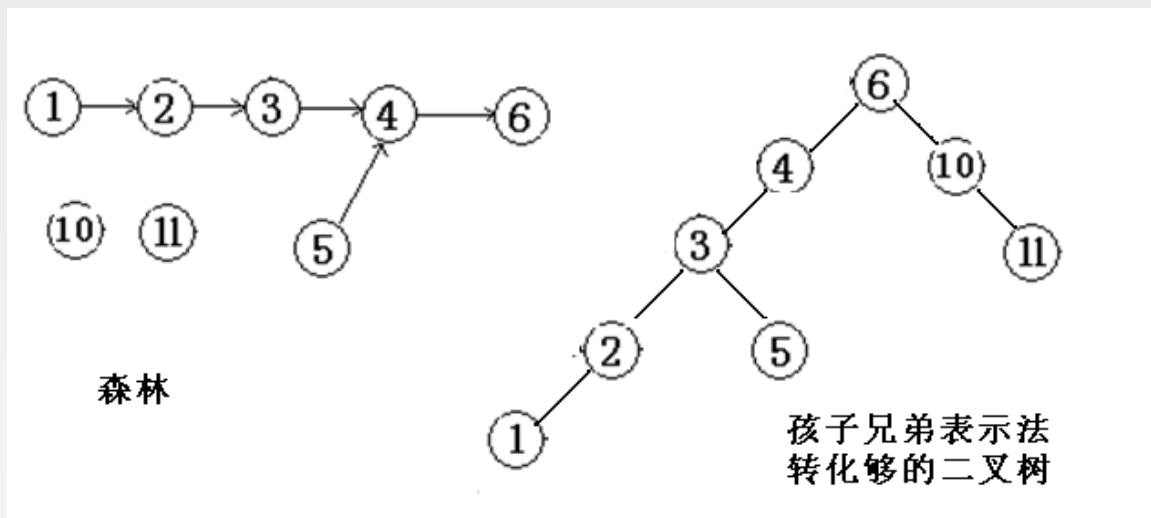


图2



→ 树型动态规划

✧ 显然这个森林可以采用树型动态规划，先将它转换成儿叉树



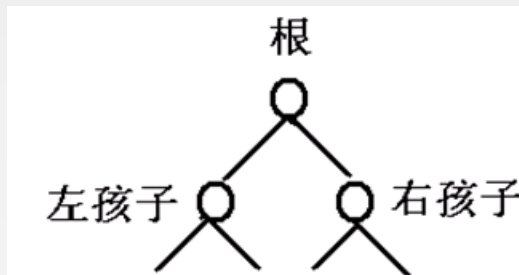
✧ 设 $f(i,j)$ 表示以 i 为根结点的二叉树分配 j 资源的最大价值

$$f(r[i], j) = \max \begin{cases} f(r[i_l], k) + f(r[i_r], j - k - w[r[i]]) + c[r[i]], & \text{根结点选} \\ f(r[i_r], j), & \text{根结点不选} \end{cases}$$



→ 总结

- ✧ 树型动态规划有一个共性，那就是它的**基本模型都是一棵树或者森林**
- ✧ 为了考虑方便，一般情况下都将这个树或森林转化为二叉树，如下图
- ✧ 然后整个问题的**最优只会涉及到左右孩子的最优**，然后考虑根结点的情况，这样化简了问题，最终很容易写出状态转移方程，从而问题得到解决。



- ✧ 另外，并不是所有的问题一定要转化为二叉树来解决，要仔细思考的就是每个结点有些什么状态，这些结点的状态与父、子结点的状态有什么联系，也就是如何由子结点的最优值推出父节点的最优值(即状态转移方程)。



➔ 练习题目

- ✧ Hzwer, 树形DP 54题
- ✧ <http://hzwer.com/category/algorithm/dp/treedp>
- ✧ <https://blog.csdn.net/txl199106/article/details/45372337>(参考学习博客)
- ✧ 练习题目:
- ✧ **P3177 [HAOI2015]树上染色**
- ✧ **P1131 [ZJOI2007]时态同步**
- ✧ **P1270 “访问” 美术馆**
- ✧ **P2016 战略游戏**

The background features a central perspective view of a tunnel-like structure formed by multiple colored rays (blue, orange, red, white, and grey) converging towards a vanishing point. The rays are set against a dark grey background. Faint, semi-transparent binary code (0s and 1s) is scattered across the scene, particularly in the upper right area. On the right side, there is a faint line graph with a yellow line and a blue line, plotted against a grid with dates on the x-axis (00-12-01, 00-01-01, 00-02-01, 00-03-01, 00-04-01).

Thank you!